SMART FILE MANAGEMENT AND ARCHIVAL AUTOMATION SYSTEM

Project Report

A Python-based automation solution for document content extraction, classification, cloud archival, duplicate detection, and audit tracking.

Core platforms	Python · Streamlit · Google Drive API · SQLite · Google Sheets API
Document analysis	PyMuPDF text extraction · Tesseract OCR · weighted phrase scoring
Key exceptions	Invalid upload · duplicate content · cloud/database failure handling

Prepared as an academic prototype. Google API integrations require credentials and explicit configuration before a live demonstration.

1. Abstract

The Smart File Management and Archival Automation System reduces manual document filing by extracting text from uploaded files, classifying content with transparent weighted phrase rules, detecting duplicates using SHA-256, storing documents in Google Drive, recording searchable metadata in SQLite, and sending audit events to Google Sheets. For scanned PDFs, the application uses Tesseract OCR when the PDF text layer is insufficient. Documents with weak or conflicting evidence are assigned to Needs Review instead of being classified with false certainty.

2. Problem Statement and Objectives

Manual document management can result in misfiled documents, duplicate copies, slow retrieval, and limited traceability. Filename-only classification is unreliable because names may be generic or meaningless. This project addresses these issues by basing classification primarily on document contents and by separating cloud file storage from metadata and audit records.

• Extract text from supported files and OCR scanned PDFs when needed.

• Classify documents into Invoices, Reports, Certificates, Other, or Needs Review.

• Reject invalid, empty, and oversized uploads.

• Detect identical file content regardless of filename.

• Store actual documents in Google Drive category folders.

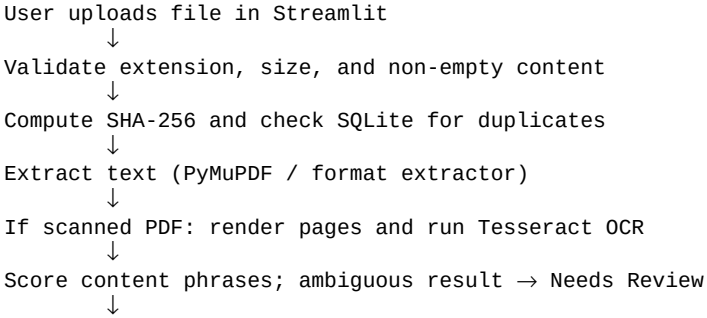
• Store metadata and status in SQLite and mirror audit events to Google Sheets.

• Record failures and use compensating cleanup when database persistence fails after a Drive upload.

3. Technology Stack and Responsibilities

Component	Responsibility
Python	Orchestrates validation, classification, duplicate checks, cloud upload, logging, and exceptions.
Streamlit	Provides the browser dashboard, upload flow, preview, archive search, and audit view.
PyMuPDF	Extracts text from PDFs and renders pages for OCR when needed.
Tesseract OCR	Recognizes text from scanned PDF pages and image uploads.
Google Drive API	Stores original uploaded documents in category folders under a shared root folder.
SQLite	Stores hashes, metadata, category, status, Drive identifiers, extraction method, and audit events.
Google Sheets API	Provides an external audit trail that can be inspected during evaluation.

4. System Architecture and Workflow



```
Upload original document to Google Drive category folder
        ↓
Store metadata and Drive link in SQLite
        ↓
Write audit event to SQLite and attempt Google Sheets sync
```

Google Drive is the file store. SQLite stores metadata, not document bytes. Google Sheets is an audit mirror. If Sheets is unavailable, the local audit record remains and successful Drive storage is not undone. If the SQLite write fails after Drive upload, the application attempts to delete the cloud file; a cleanup failure is written to the application log for manual reconciliation.

5. Classification Method

The classifier applies weighted phrase rules to extracted text. Examples for invoices include “invoice number”, “amount due”, “tax invoice”, and “GSTIN”. Report evidence includes “executive summary”, “methodology”, “findings”, and “conclusion”. Certificate evidence includes “this is to certify”, “certificate number”, and “has successfully completed”. Filename clues receive a small weight so document content has greater influence.

The system assigns Invoices, Reports, or Certificates when the strongest score is sufficiently high and separated from competing categories. Weak or conflicting evidence is assigned to Needs Review. The displayed confidence is a transparent heuristic score and is not a calibrated probability or a trained machine-learning prediction. Classification quality depends on document layout, wording, OCR quality, and the configured phrase list.

6. Exception Handling

Exception 1 — Invalid file

The backend rejects unsupported extensions, empty files, oversized files, and PDF files without a valid PDF header. It records INVALID_FILE and does not create a successful archive record.

Exception 2 — Duplicate document

SHA-256 is calculated from file bytes and compared against existing SQLite records. A repeated upload is rejected even if the filename changes.

Drive/API failure

A failed upload is recorded as an error. The app does not silently save the file to a local folder or claim that archival succeeded.

Database failure after cloud upload

The app attempts a compensating delete of the Drive object. If deletion fails, the application log records the failure so the file can be reconciled manually.

Google Sheets failure

The event remains in SQLite and logs; Sheets sync status is surfaced to the user without undoing successful archival.

7. Testing Strategy and Results

The automated suite uses a fake Drive adapter to test the workflow without live credentials. Tests cover content classification with generic filenames, invoice and certificate evidence, OCR for scanned PDFs, ambiguous content, invalid extension, empty files, duplicate content, simulated Drive outage, and size limits.

Test	Expected outcome	Result in build environment
Generic report PDF	Content classified as Reports	Passed
Invoice/certificate PDFs	Correct category from content evidence	Passed

Test	Expected outcome	Result in build environment
Scanned PDF	OCR extracts text and identifies certificate	Passed
Ambiguous PDF	Needs Review	Passed
Invalid/empty/oversized	Rejected and logged	Passed
Duplicate bytes	Second upload rejected	Passed
Simulated Drive outage	Error logged; no successful document row	Passed

All 10 automated unit tests passed in the build environment. The tests use a simulated Drive client; they do not verify live Google Drive or Google Sheets credentials. A manual integration test must be performed after the service account and shared folders are configured.

8. Setup and Demonstration

Install the Python dependencies from requirements.txt, install Tesseract OCR on the host computer, configure a Google Cloud service account, enable Drive and Sheets APIs, share a Drive root folder and audit spreadsheet with the service account, then set GOOGLE_DRIVE_ROOT_FOLDER_ID, GOOGLE_SHEETS_SPREADSHEET_ID, and GOOGLE_SERVICE_ACCOUNT_JSON before launching Streamlit.

The demonstration should show: problem statement; architecture; classification of a PDF with a generic filename; successful upload and link to Google Drive; invalid-file handling; duplicate detection; Google Sheets audit row; and future enhancements.

9. Limitations and Future Scope

• The classifier is rules-based and may not understand unusual or multilingual documents.

• OCR accuracy depends on scan resolution, skew, contrast, and language configuration.

• A Google service account must be granted access to the Drive folder and audit spreadsheet.

• The prototype does not yet include user authentication, role-based access, malware scanning, or a retention policy.

• Future work could add multilingual OCR, an AI-based classifier, human feedback to improve rules, approval workflows, notifications, and production-grade monitoring.

10. Conclusion

The upgraded prototype demonstrates an end-to-end automation flow across a web interface, document extraction/OCR, cloud file storage, a relational metadata database, and an external audit spreadsheet. It addresses filename-only classification and local-only storage limitations while preserving explicit exception handling and auditability. Before submission, the student should run the project locally, configure the live Google integrations, capture their own demonstration evidence, and be prepared to explain the code and trade-offs.